🎓 جزوه طلایی سیستم‌عامل

فصل ۱۰: حافظه مجازی - بخش سوم (کپی هنگام نوشتن و جایگزینی صفحه)

۱. تکنیک کپی هنگام نوشتن (Copy-on-Write / COW)

چطور می‌توان یک پردازه جدید را با سرعت نور ایجاد کرد؟ با تنبلی هوشمندانه!

مشکل روش قدیمی (fork سنتی):

در گذشته، وقتی دستور fork() صدا زده می‌شد، سیستم‌عامل تمام صفحات والد (Parent) را دقیقاً برای فرزند (Child) کپی می‌کرد.

> **عیب:** اگر فرزند بلافاصله دستور exec() را اجرا کند (که معمولاً همینطور است)، تمام آن کپی کردن‌ها بیهوده بود!

راهکار طلایی (COW):

به جای کپی کردن، به والد و فرزند اجازه می‌دهیم **صفحات را به اشتراک بگذارند**، اما این صفحات را به عنوان «کپی هنگام نوشتن» علامت‌گذاری می‌کنیم.

- **سناریو ۱ (فقط خواندن):** تا زمانی که هر دو فقط می‌خوانند، از همان یک نسخه فیزیکی مشترک استفاده می‌کنند (صرفه‌جویی عالی در حافظه و زمان).
- **سناریو ۲ (نوشتن):** به محض اینکه یکی از آن‌ها (مثلاً فرزند) بخواهد در یک صفحه مشترک بنویسد، سیستم‌عامل متوجه می‌شود و **فقط همان یک صفحه** را کپی می‌کند تا فرزند نسخه اختصاصی خودش را داشته باشد.

✅ مزایا:

1. سرعت بسیار بالا در ایجاد پردازه.
2. صرفه‌جویی چشمگیر در مصرف RAM.

⚡ نکته کنکوری: vfork()

یک نسخه خطرناک‌تر و سریع‌تر از fork است!

- در vfork، پردازه والد **معلق (Suspend)** می‌شود و فرزند مستقیماً از فضای آدرس والد استفاده می‌کند (هیچ کپی‌برداری حتی COW هم انجام نمی‌شود).
- **خطر:** اگر فرزند دیتای والد را تغییر دهد، دیتای والد خراب می‌شود!
- **کاربرد:** فقط زمانی که فرزند بلافاصله exec می‌کند.

۲. جایگزینی صفحه (Page Replacement)

وقتی رم پر شد، چه کسی باید برود؟

مشکل تخصیص بیش از حد (Over-allocation):

با حافظه مجازی، می‌توانیم درجه چندبرنامگی را بالا ببریم. مثلاً ۶ پردازه داریم که هرکدام ۱۰ صفحه دارند (مجموع ۶۰ صفحه)، اما کل رم ما فقط ۴۰ قاب (Frame) دارد. اگر همه پردازه‌ها ناگهان تمام صفحاتشان را بخواهند، **قاب آزاد (Free Frame) تمام می‌شود.**

راه‌حل: جایگزینی صفحه

به جای بستن برنامه کاربر، سیستم‌عامل یک صفحه که فعلاً استفاده نمی‌شود را پیدا کرده، آن را از رم بیرون می‌اندازد (Page Out) و صفحه جدید را جای آن می‌آورد.

🔄 ۳. الگوریتم پایه جایگزینی صفحه (۴ مرحله مهم)

وقتی خطای صفحه رخ می‌دهد و قاب آزاد نداریم:

1. **انتخاب قربانی (Victim):** سیستم‌عامل با استفاده از یک الگوریتم جایگزینی، صفحه‌ای را برای حذف انتخاب می‌کند.
2. **خروج قربانی (Page Out):** صفحه قربانی را در دیسک (Swap Space) می‌نویسیم.
    - *سپس جداول را آپدیت می‌کنیم (بیت آن را Invalid می‌کنیم).*
1. **ورود صفحه جدید (Page In):** صفحه مورد نظر را از دیسک به همان قابی که خالی شده می‌آوریم.
2. **شروع مجدد:** دستور را دوباره اجرا می‌کنیم.

💎 ۴. بیت تغییر (Modify Bit / Dirty Bit) - بهینه‌سازی حیاتی

در مرحله ۲ بالا، آیا همیشه لازم است قربانی را در دیسک بنویسیم؟ خیر!

- **بیت تغییر (M):** یک بیت سخت‌افزاری است که اگر صفحه‌ای تغییر کند (نوشته شود)، مقدارش ۱ می‌شود.
- **اگر بیت M=0 باشد (Clean):** یعنی صفحه از وقتی لود شده تغییری نکرده. پس نسخه روی دیسک هنوز معتبر است. **نیاز به نوشتن در دیسک نیست** (فقط روی آن را بازنویسی می‌کنیم).
- **اگر بیت M=1 باشد (Dirty):** صفحه تغییر کرده و باید حتماً روی دیسک نوشته شود.

✅ **تاثیر:** این تکنیک زمان سرویس‌دهی به خطای صفحه را **نصف** می‌کند (چون یک I/O نوشتن حذف می‌شود).

۵. ارزیابی الگوریتم‌ها (Reference String)

چطور بفهمیم کدام الگوریتم جایگزینی بهتر است؟ با شمردن تعداد خطاهای صفحه (Page Faults).

- **رشته ارجاع (Reference String):** دنباله‌ای از شماره صفحات که برنامه به آن‌ها مراجعه می‌کند. (آدرس‌های کامل را به شماره صفحه تبدیل می‌کنیم).
- **هدف:** الگوریتمی که کمترین خطای صفحه را داشته باشد.
- **قاعده کلی:** معمولاً با افزایش تعداد قاب‌های فیزیکی، تعداد خطاهای صفحه کاهش می‌یابد (البته استثنایی به نام *Belady* وجود دارد که بعداً می‌خوانیم).

مثال تبدیل آدرس به رشته ارجاع: اگر اندازه صفحه ۱۰۰ بایت باشد: آدرس‌ها: 0100, 0432, 0101, 0612 شماره صفحات (با تقسیم بر ۱۰۰): 1, 4, 1, 6

📝 خلاصه نکات کلیدی:

- **COW:** اشتراک صفحات بین والد و فرزند تا لحظه نوشتن.
- **vfork:** والد معلق می‌شود، فرزند روی حافظه والد سوار می‌شود (خیلی سریع اما خطرناک).
- **Dirty Bit:** اگر صفحه‌ای تغییر نکرده باشد (Clean)، هنگام تعویض نیازی به نوشتن در دیسک ندارد (کاهش سربار I/O).
- **وظیفه الگوریتم جایگزینی:** انتخاب بهترین قربانی برای کمترین نرخ خطا در آینده.